IIT MADRAS · SOFTWARE ENGINEERING · SEPT 2026

Milestone 2 Report

Scheduling, Design and Frontend

Student Mentoring and Career Support Platform

Problem Statement 2

Team 032 – Himalayan Coders

Himanshu Rathor · Ankur Maurya · Hemendra Kumar Sharma
Adnan Mustafa · Bojannagari Pavan Kumar Reddy

1. Project Schedule and Sprint Plan

1.1 Scheduling tool

We use **Jira** (project key KAN) to plan and track our work. An epic groups the work of one milestone or sprint. Each task has one owner, a start date and a due date, and moves across the board columns: To Do, In Progress, In Review and Done. The Jira **Timeline** is our Gantt chart and the Jira **Board** is our Kanban board. Because every task has an owner, both views show what each member does.

1.2 Team roles

Member	Roles
Himanshu Rathor	Scrum Master, Product Manager
Ankur Maurya	Backend Developer, Frontend Developer
Hemendra Kumar Sharma	Frontend Developer, Backend Developer
Adnan Mustafa	Tester, Code Reviewer
Bojannagari Pavan Kumar Reddy	Code Reviewer, Tester

1.3 Overall schedule

Phase	Dates	Focus
Milestone 1	until 25 Oct	User requirements
Milestone 2	26 Oct – 1 Nov	Scheduling, design, frontend pages
Sprint 1 (Milestone 3)	2 Nov – 22 Nov	APIs, test cases and user testing for the core flow
Sprint 2 (Milestone 4)	23 Nov – 6 Dec	APIs, test cases and user testing for the rest of the flow
Milestone 5	7 Dec – 20 Dec	Final report, demo video, working prototype

1.4 Sprint 1 (2 Nov – 22 Nov): core client-mediated flow

Jira task	User stories	Dates
Registration and login APIs	US-S01, US-M01	2 – 8 Nov
Student and mentor profile APIs	US-S02, US-M02, US-C05	6 – 12 Nov
Admin dashboard and role-based access APIs	US-C01, US-C06	9 – 15 Nov
Match creation APIs	US-C02	12 – 18 Nov
Test cases, pytest and Swagger YAML	all of the above	9 – 20 Nov
User feedback session and Sprint 2 plan	–	20 – 22 Nov

1.5 Sprint 2 (23 Nov – 6 Dec): confirmation, tracking and student views

Jira task	User stories	Dates
Match confirmation and notification APIs	US-C03	23 – 29 Nov
Student goals, preferences and availability APIs	US-S03 to US-S06	23 – 29 Nov
Match status management and overview APIs	US-C04, US-C07	26 Nov – 2 Dec
Student connection view, readiness, concern, feedback and status APIs	US-S07 to US-S11	28 Nov – 3 Dec
Test cases, pytest and Swagger YAML	all of the above	30 Nov – 5 Dec
User feedback session and final-phase plan	–	4 – 6 Dec

All 21 user stories from Milestone 1 are covered across the two sprints.

1.6 Iterations

Each sprint is split into weekly iterations. Every sprint ends with a user feedback session, and its findings shape the next sprint. Code Reviewers review each task and Testers test it before it moves to Done.

1.7 Gantt chart

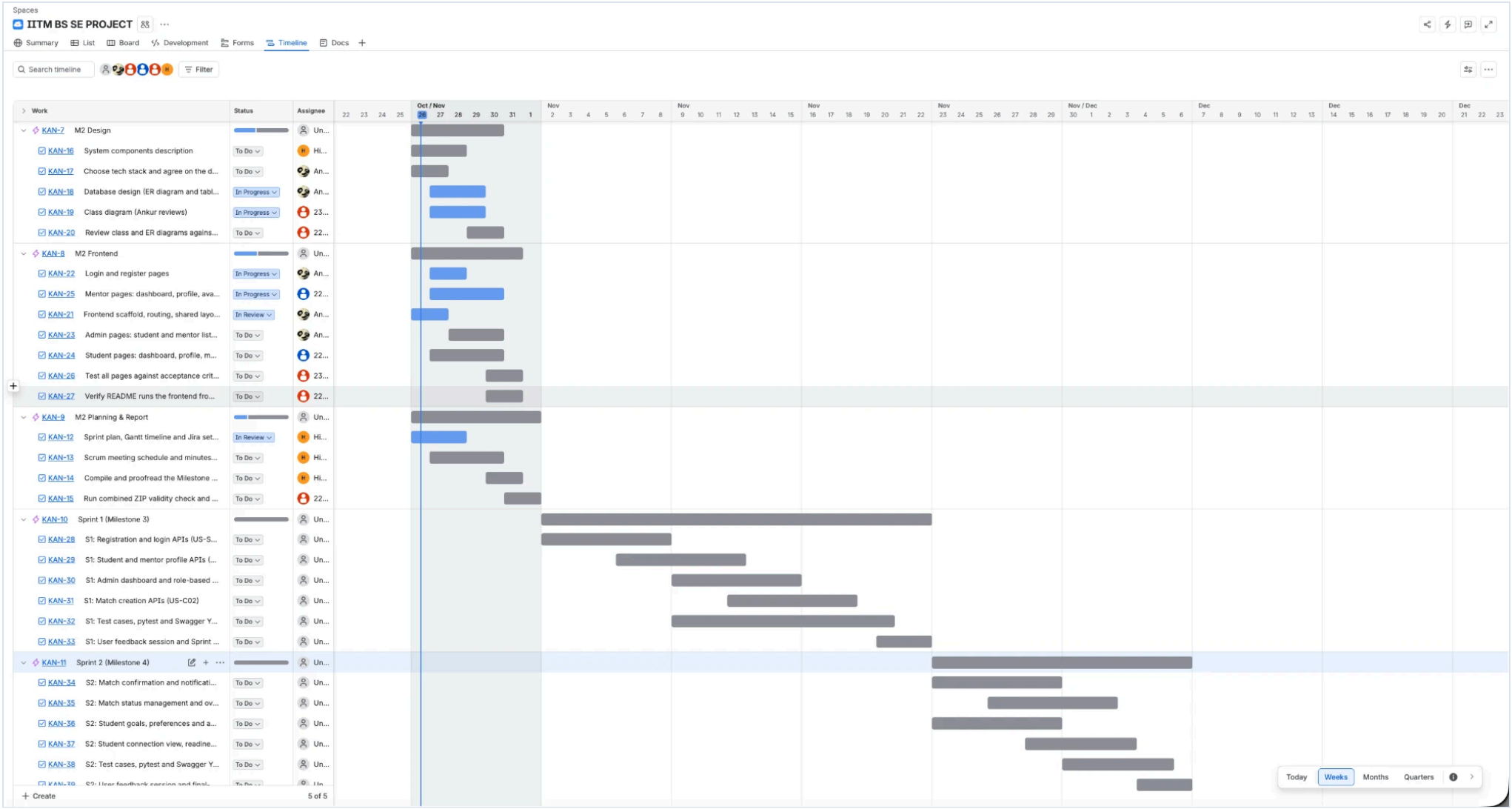


Figure A. Jira Timeline (Gantt) with the epics expanded, from 26 Oct to December.

1.8 Kanban board

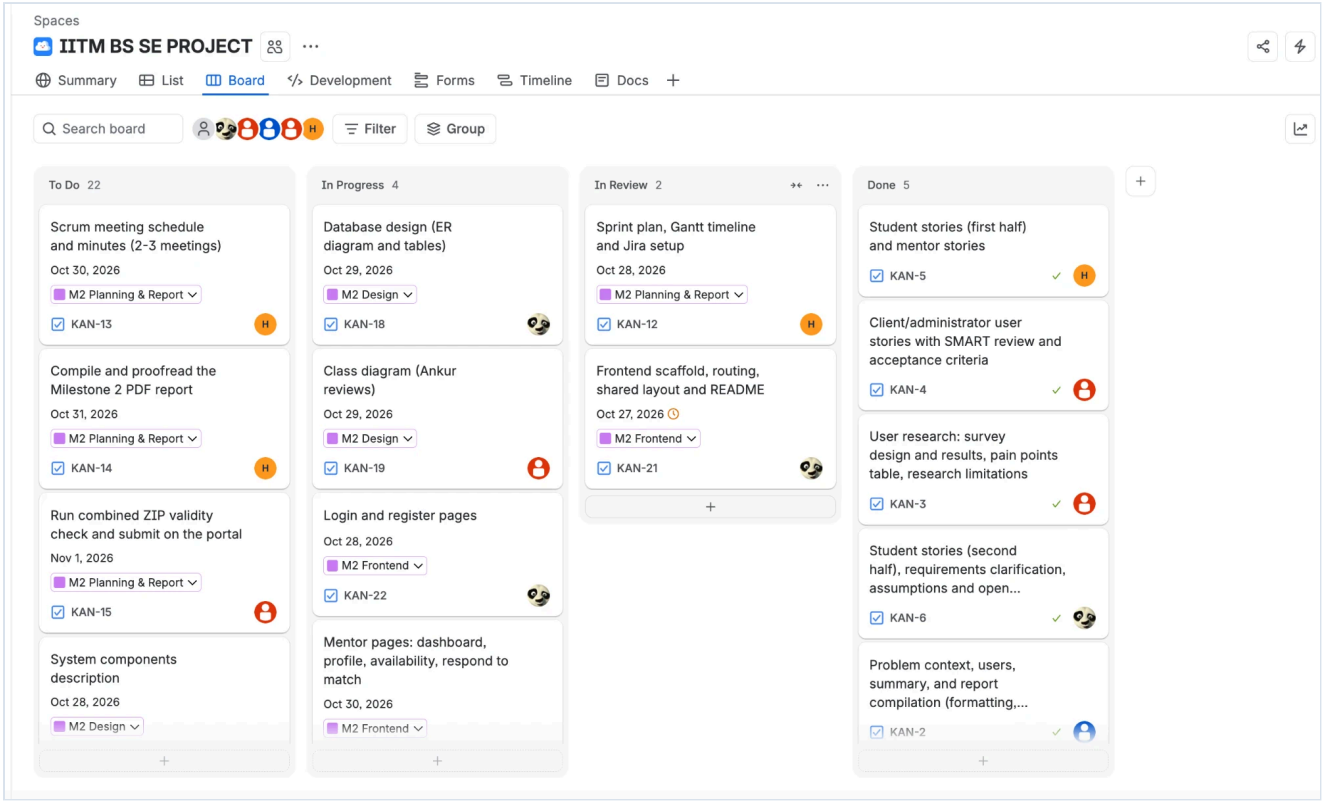


Figure B. Jira Kanban board with the Milestone 2 tasks across To Do, In Progress, In Review and Done.

2. Scrum Meetings Schedule and Minutes

Scrum meetings: held about twice a week on Google Meet and led by the Scrum Master. Each meeting covers what was done, what comes next and any blockers. Two meetings were held in Milestone 2.

Minutes from Scrum Meeting 1

Date and time	27 October 2026, 6:00 – 6:57
Mode	Google Meet
Attendees	Ankur, Adnan, Hemendra, Himanshu, Pavan
Agenda	Frontend design; database schema; workflow for matching students to mentors.

Discussion. The team discussed the overall design of the frontend and the database schema of the platform. The team also discussed how a student is matched to a mentor. Each match will show a status of Proposed, Active, Completed or Cancelled.

Decision. The match status values are fixed as Proposed, Active, Completed and Cancelled.

Next meeting agenda. Continue discussing the frontend design in more detail, and decide how the dashboards will look for the Admin, the Student and the Mentor.

Minutes from Scrum Meeting 2

Date and time	31 October 2026, 7:00 – 7:45 PM
Mode	Google Meet
Attendees	Ankur, Adnan, Hemendra
Agenda	Scrum call 2 minutes of meeting.

#	Discussed	Action (owner)
1	Jira tasks and dashboard.	The team reviews their tasks and makes changes.
2	Changes in the ER diagram.	Ankur (completed)
3	Milestone 2 report review.	Adnan
4	User story US-C03 clarification: the design was modified as per this story.	Ankur (completed)
5	GitHub Milestone 2 review.	The team, as per the tasks assigned in Jira

3. Design of Components

Our platform is **client-mediated**. Students and mentors each keep their own profile, but they cannot see each other. The admin (our client) looks at both sides, proposes a match and follows it until it ends.

Component	What it does
Login and roles	Registration and login for three roles: student, mentor and admin. Each role sees only its own pages.
Student module	The student keeps their goals, skills, wanted topics, preferred format, duration and availability. The student sees the status of their match, accepts or declines the mentor proposed by the admin, gives feedback and can raise a concern.
Mentor module	The mentor keeps their qualifications, topics, bio, availability slots and the maximum number of students they can mentor at the same time. The mentor accepts or declines match requests and sees their active mentees.
Admin module	The admin views student profiles, reviews and verifies mentor profiles, sees recommended matches, proposes a match, confirms or cancels it, changes its status, and follows concerns and feedback.
Matching and recommendation	Gives each student and mentor pair a score out of 100 and shows the best mentors to the admin, with reasons. It only suggests. The admin decides. See section 3.1.
Match workflow	Controls the status of a match: proposed, active, completed or cancelled. A match becomes active only when the admin confirms it after both sides have accepted. Every change is saved in a history.
Notifications	Tells the mentor and the student when a match is proposed. Tells the admin when someone accepts or declines. Tells both people when the admin confirms or cancels, and what the next step is.
Database	Stores users, profiles, topics, mentor availability, matches with their scores, status history, feedback, concerns and notifications.

How a match moves: the admin proposes a match, and the mentor and the student are both notified. Each of them accepts or declines. When **both have accepted**, the admin can confirm the match and it becomes active. The admin can also cancel it at any time. Later an active match is completed or cancelled. If someone declines, the admin cancels the match and can propose a new one.

3.1 How matches are recommended

The admin should not have to compare every student with every mentor. The platform makes a short list. The method is simple. It adds points for two facts the admin can check:

Part	Points	How it is worked out

Topic overlap	70	Students and mentors choose topics from the same list. The score is the share of the student's wanted topics that the mentor also covers.
Mentor experience	30	Years of experience, counted up to 10 years.

Pairs that are not shown: the mentor is not verified, the mentor already has as many students as the maximum they set, or the student already has an open match. Free places give no points; they only decide whether a pair is allowed.

Example. Neha wants Product Management and Career Switching. Priya covers both and has 8 years of experience, so the score is 70 + 24 = **94**. Rohan wants Interview Preparation and Web Development. Priya covers one of the two (35) and Dr. Meera covers none (0). With their experience, Priya scores **59** and Dr. Meera **30**, which is shown as a weak fit (below 40). Each row also shows reasons, such as "Shared topics: Interview Preparation".

Mentor capacity. Each mentor sets "maximum students at the same time" (1 to 10, default 3). Proposed and active matches use a place. A proposal that would go over the maximum is refused. A cancelled or completed match frees a place.

Admin dashboard. The overview is kept simple. A thin line at the top shows the number of matches in each status and the number of students and mentors. The first card, "Needs your attention", lists six things on one line each, with a count and a button that goes to the right page: matches ready to confirm, proposals declined by one side, mentors waiting to be verified, students waiting for a match, proposals waiting more than 3 days, and open concerns. The second card shows the best mentor for each waiting student with a Propose button. The last card has four tabs, shown one at a time: open concerns, student feedback, mentor capacity and recent activity. Everything the admin needs is on this one page, and nothing is hidden. When a pair is proposed, its score and reasons are saved with the match. The server version of this logic is built in Sprint 1 (match creation APIs).

4. Class Diagram

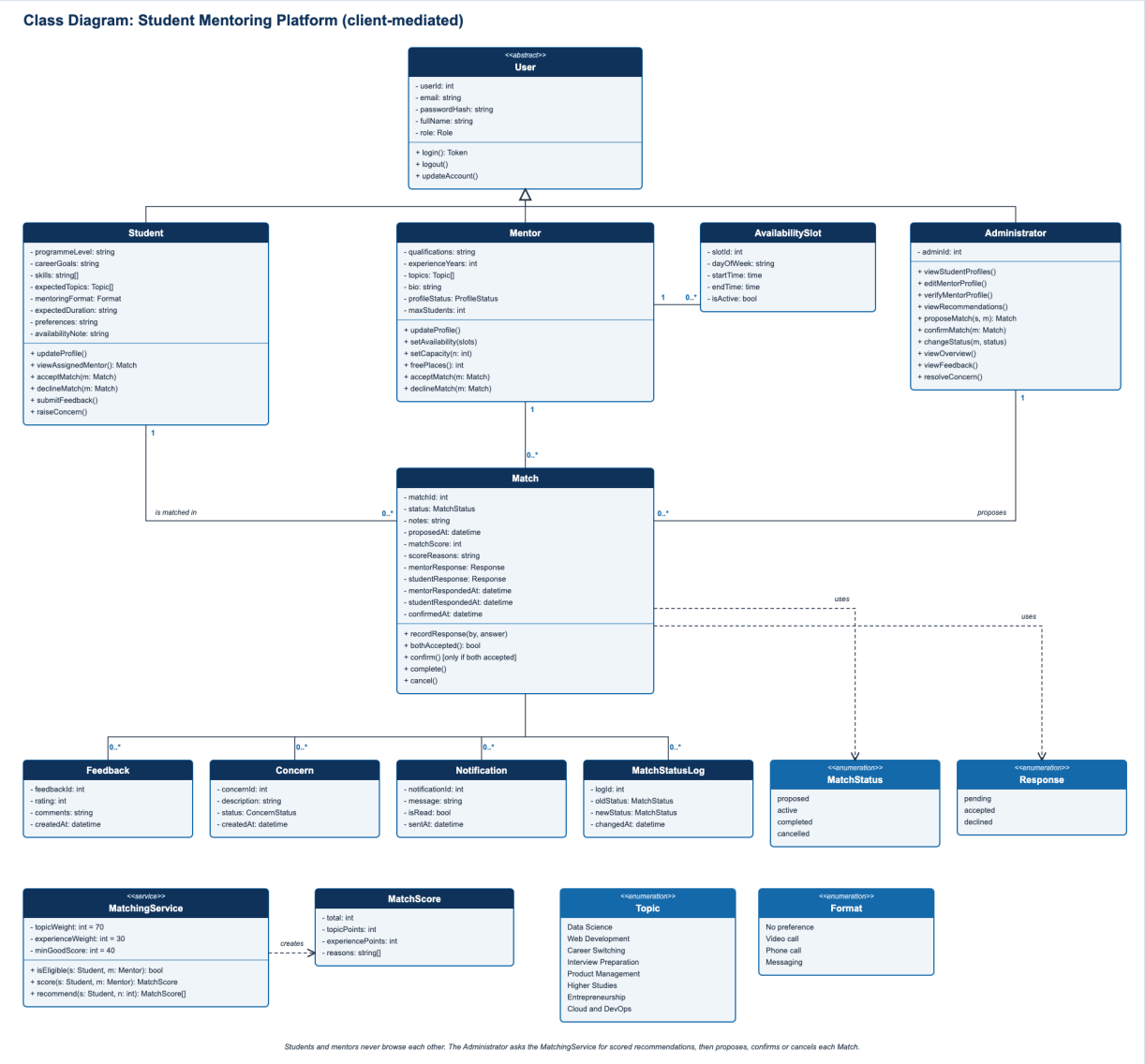


Figure 1. Class diagram. Student, Mentor and Administrator are kinds of User. Every Match is proposed by an Administrator and links one Student and one Mentor. The MatchingService scores pairs.

5. Database Design

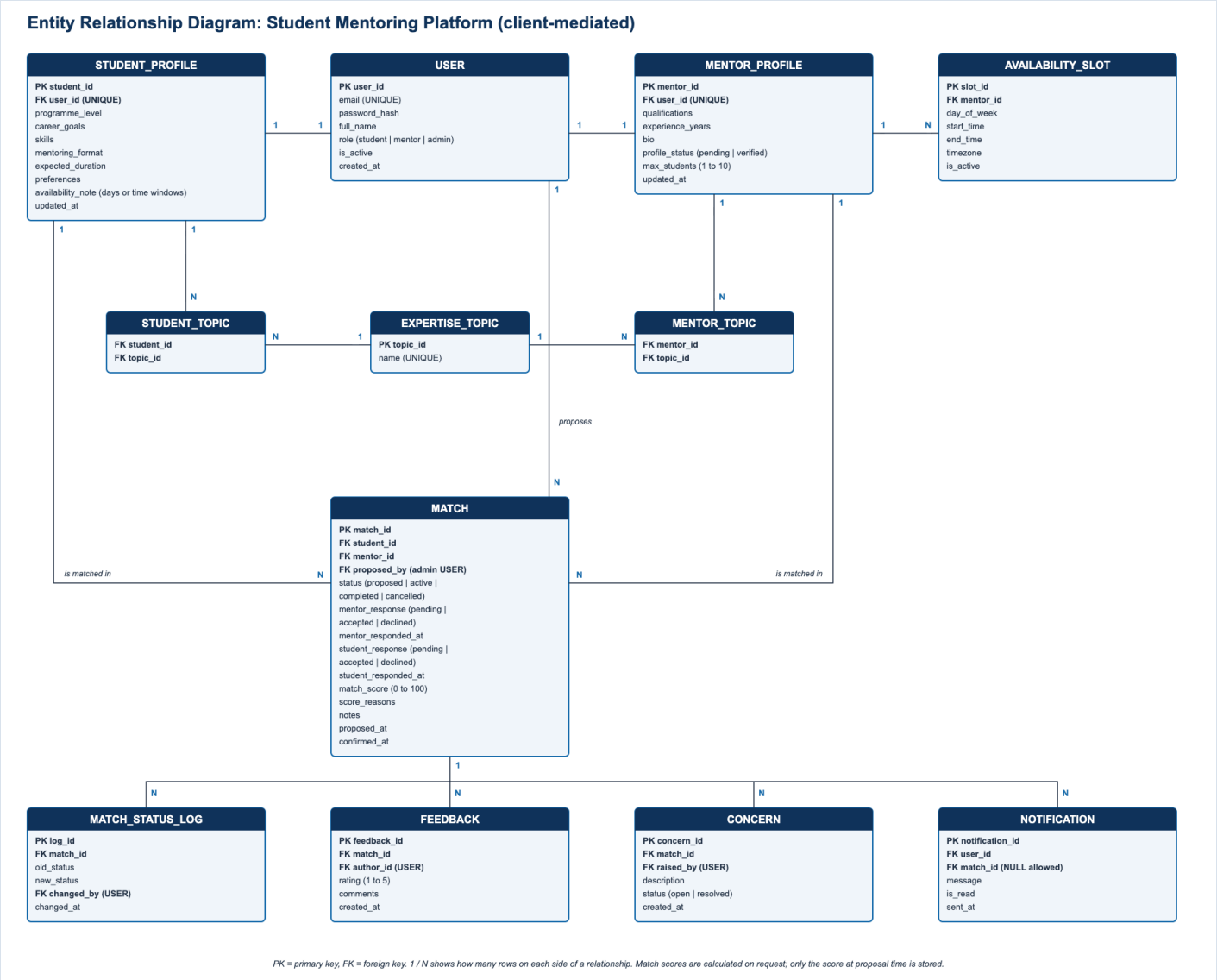


Figure 2. Entity relationship diagram.

Table	What it stores
USER	One row for each account. The email is unique. The role is student, mentor or admin.
STUDENT_PROFILE	Goals, skills, preferred format and duration, preferences and availability. One per student.
MENTOR_PROFILE	Qualifications, experience, bio, verification status and the maximum number of students (`max_students`). One per mentor.
AVAILABILITY_SLOT	The weekly time slots of a mentor. A mentor can have many.
EXPERTISE_TOPIC	The shared list of topics, for example Data Science or Web Development.
STUDENT_TOPIC, MENTOR_TOPIC	Which topics a student wants and which topics a mentor covers.
MATCH	One student and one mentor. It stores the admin who proposed it, the status, the mentor's and the student's answers with dates, the match score with its reasons, and the confirmation date.
MATCH_STATUS_LOG	The history of every status change: who changed it and when.
FEEDBACK	The rating and comments of a student about a match.
CONCERN	A problem raised about a match. It stays open until the admin resolves it.
NOTIFICATION	Messages for users, sometimes linked to a match.

6. Frontend Pages and Screenshots

The frontend is a **Vue 3** app (built with Vite) with 11 linked pages. Pages are connected with Vue Router, and each role is redirected to its own pages. Sample data is kept in the browser, so the whole flow can be tried: the admin proposes a match, both sides accept, and the admin confirms. There is no backend yet. The data calls are in one folder (`src/services`), so in Milestones 3 and 4 the real APIs replace them and the pages stay as they are.

Page	Used by	Purpose
login, register	everyone	Log in and create an account
student	student	Dashboard with match status and notifications
student-profile	student	Goals, skills, wanted topics, format, duration, availability
student-mentor	student	Status, next step, proposed mentor to accept or decline, feedback, concerns
mentor	mentor	Accept or decline requests, see active mentees

mentor-profile	mentor	Background, topics, maximum students, availability
admin	admin	Things that need attention, recommended matches, and tabs for concerns, feedback, capacity and activity
admin-students	admin	View, search and filter student profiles
admin-mentors	admin	Review, edit and verify mentors, availability and capacity
admin-matches	admin	Recommended matches, propose, confirm or cancel, change status

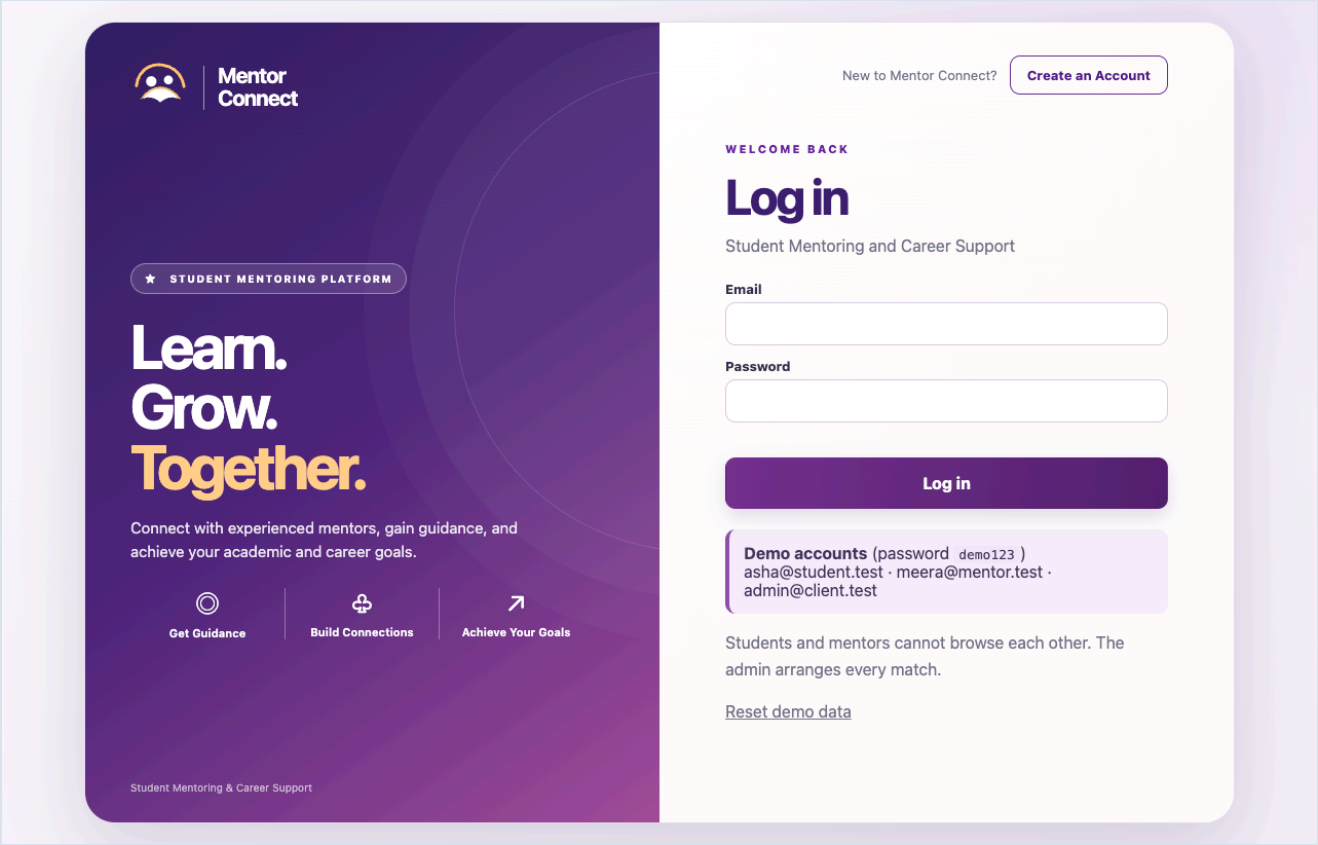


Figure 3. Login page with demo accounts.

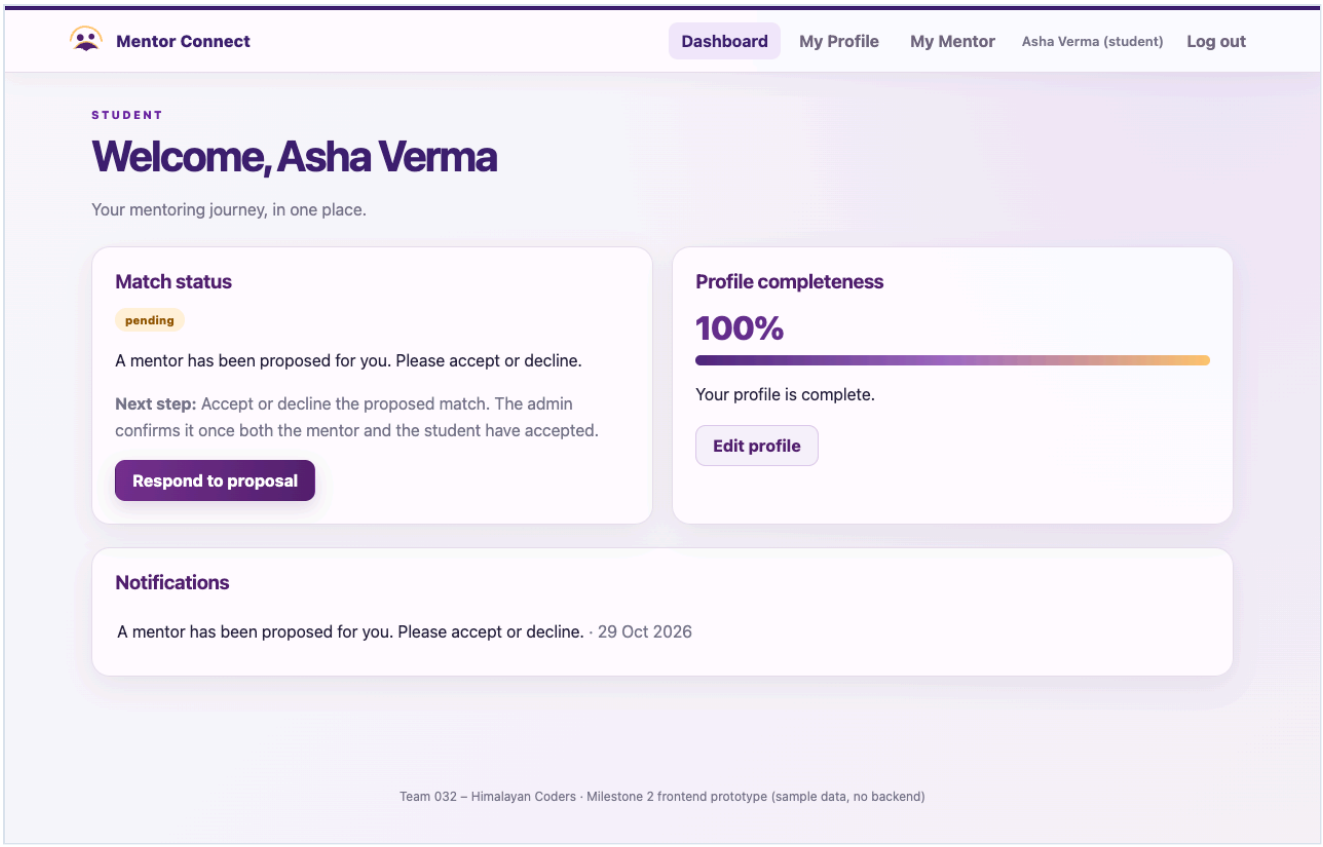


Figure 4. Student dashboard.

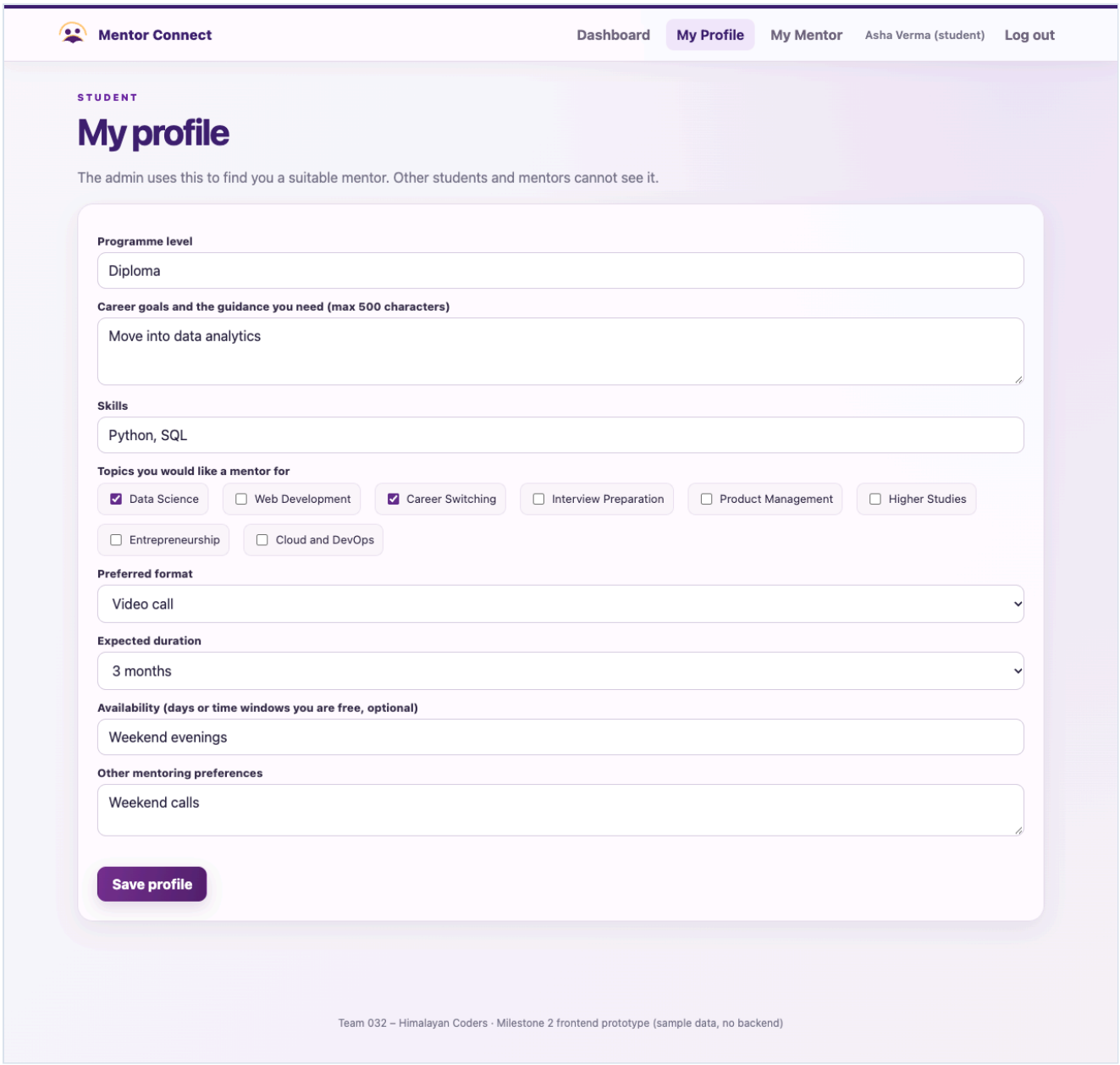


Figure 5. Student profile with wanted topics, format, duration and availability.

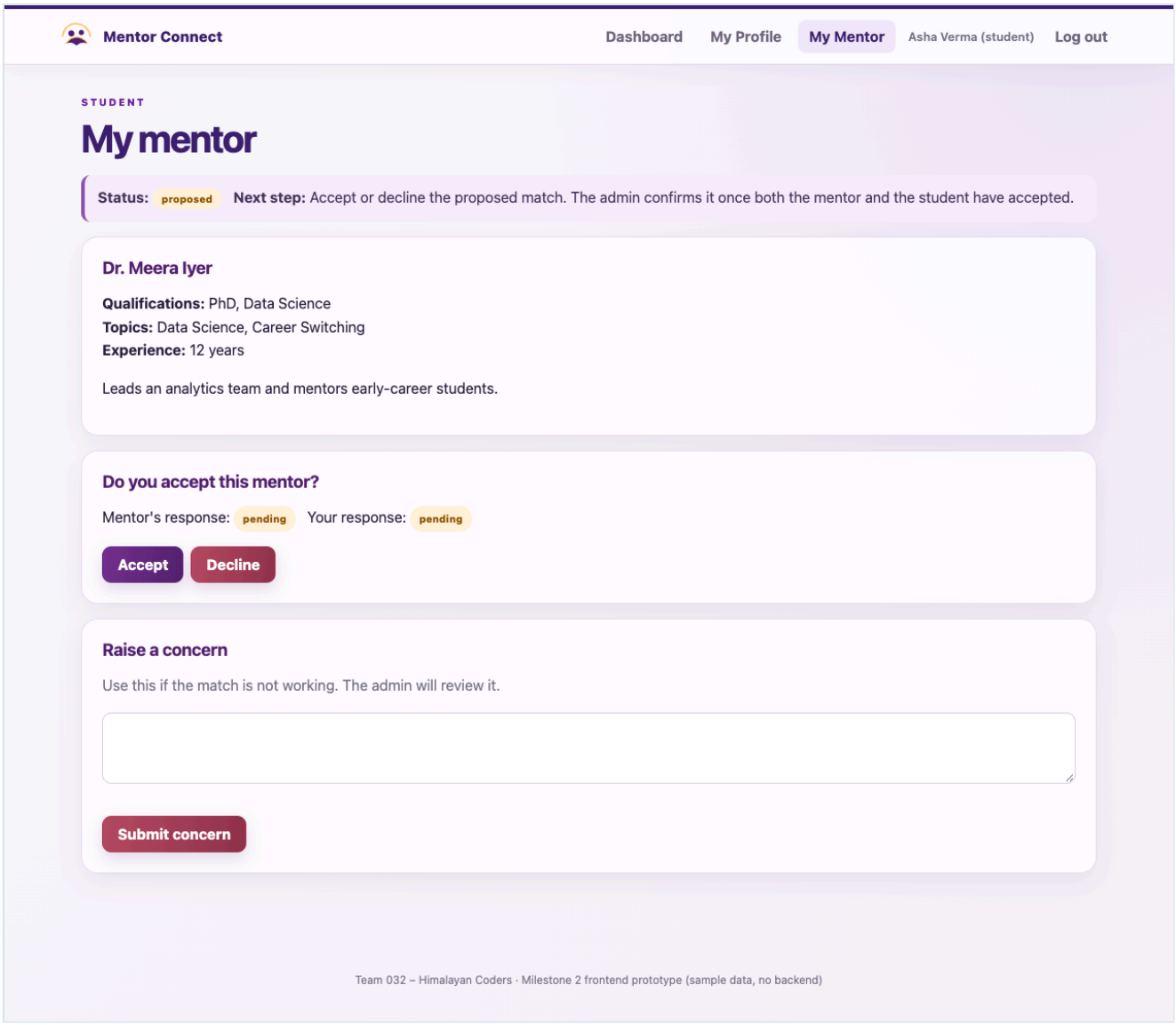


Figure 6. Student's mentor page while a match is proposed, with the Accept and Decline buttons.

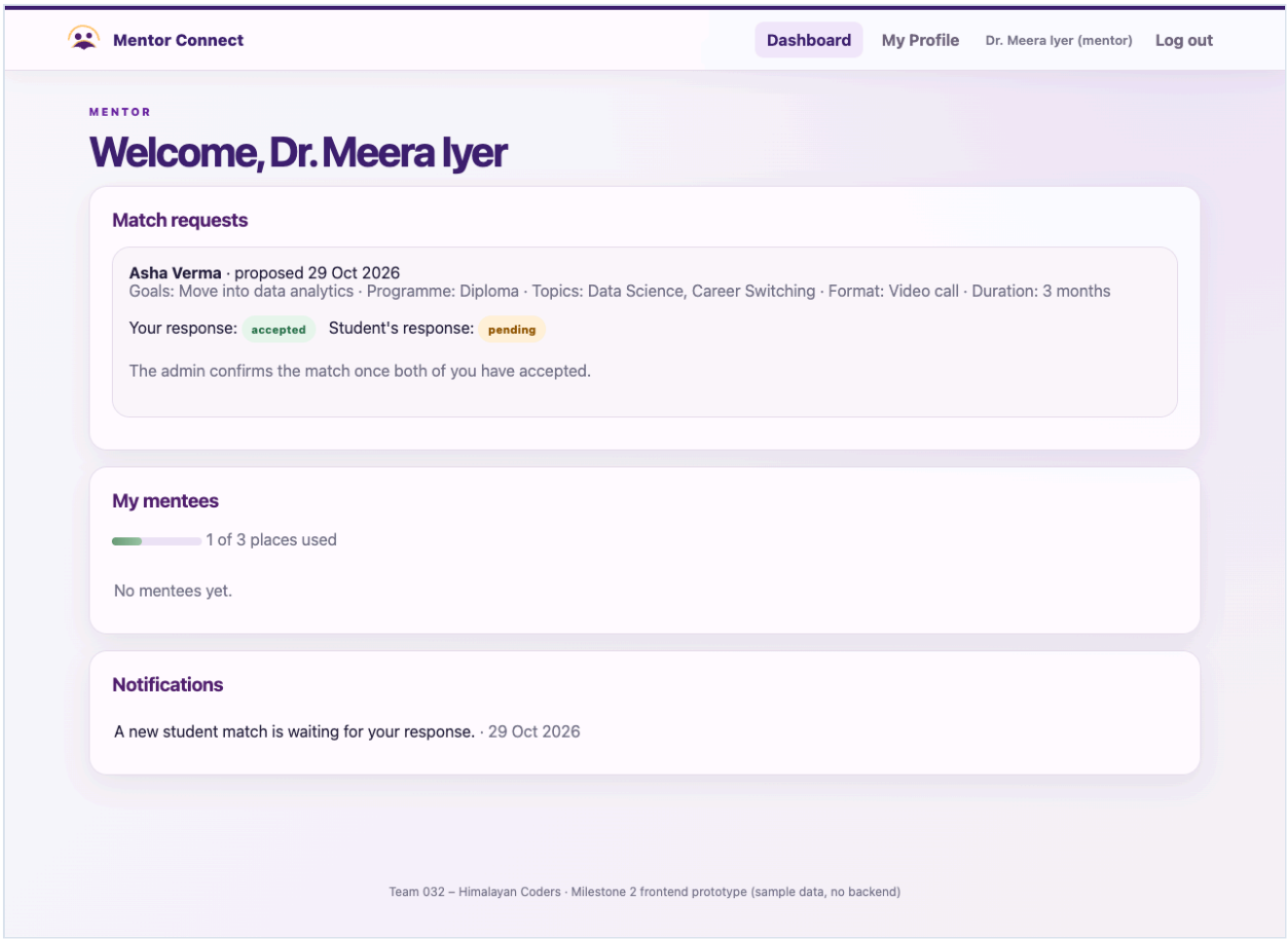


Figure 7. Mentor dashboard with a match request. It shows the mentor's and the student's answers.

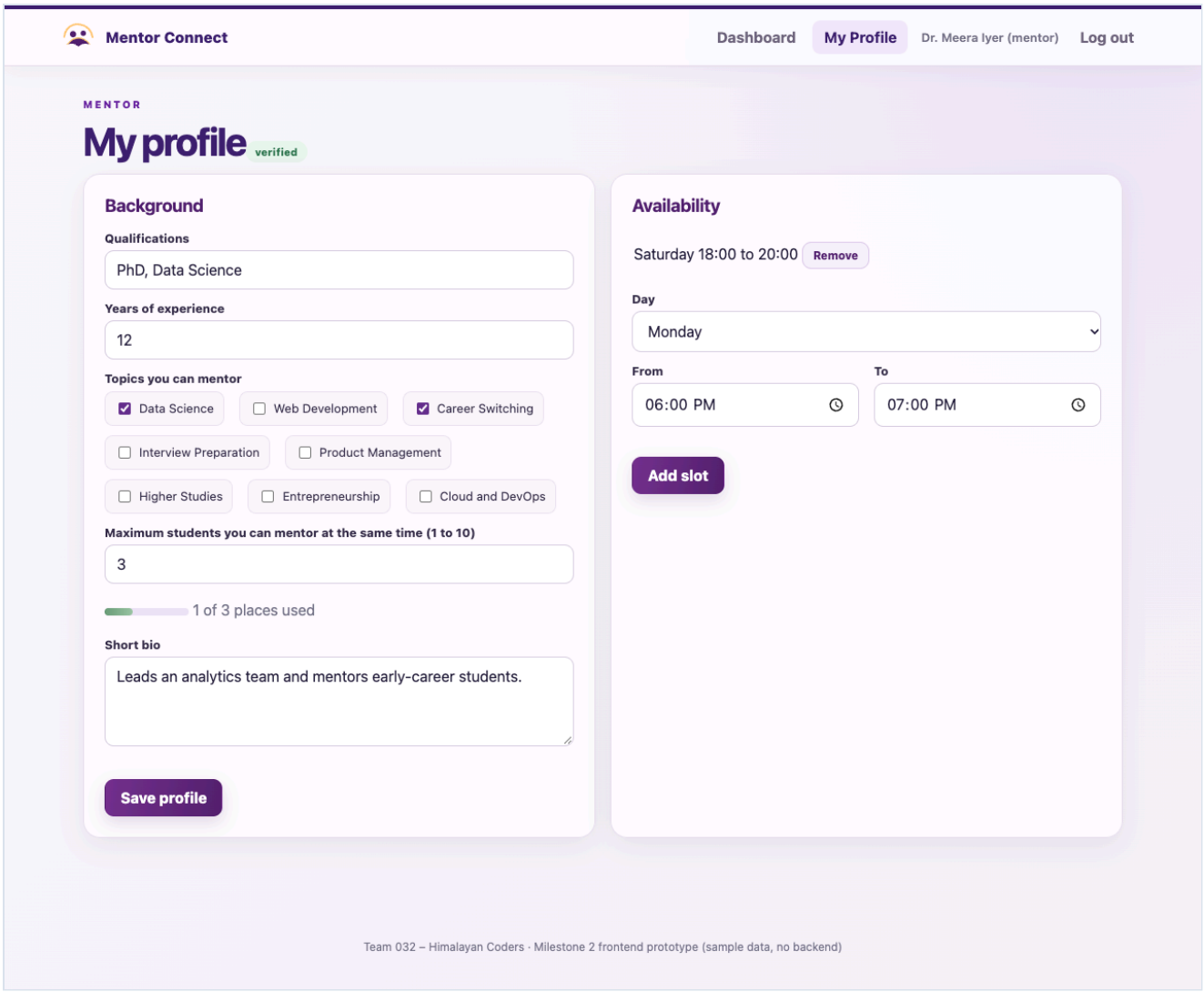


Figure 8. Mentor profile with topics, maximum students and availability slots.

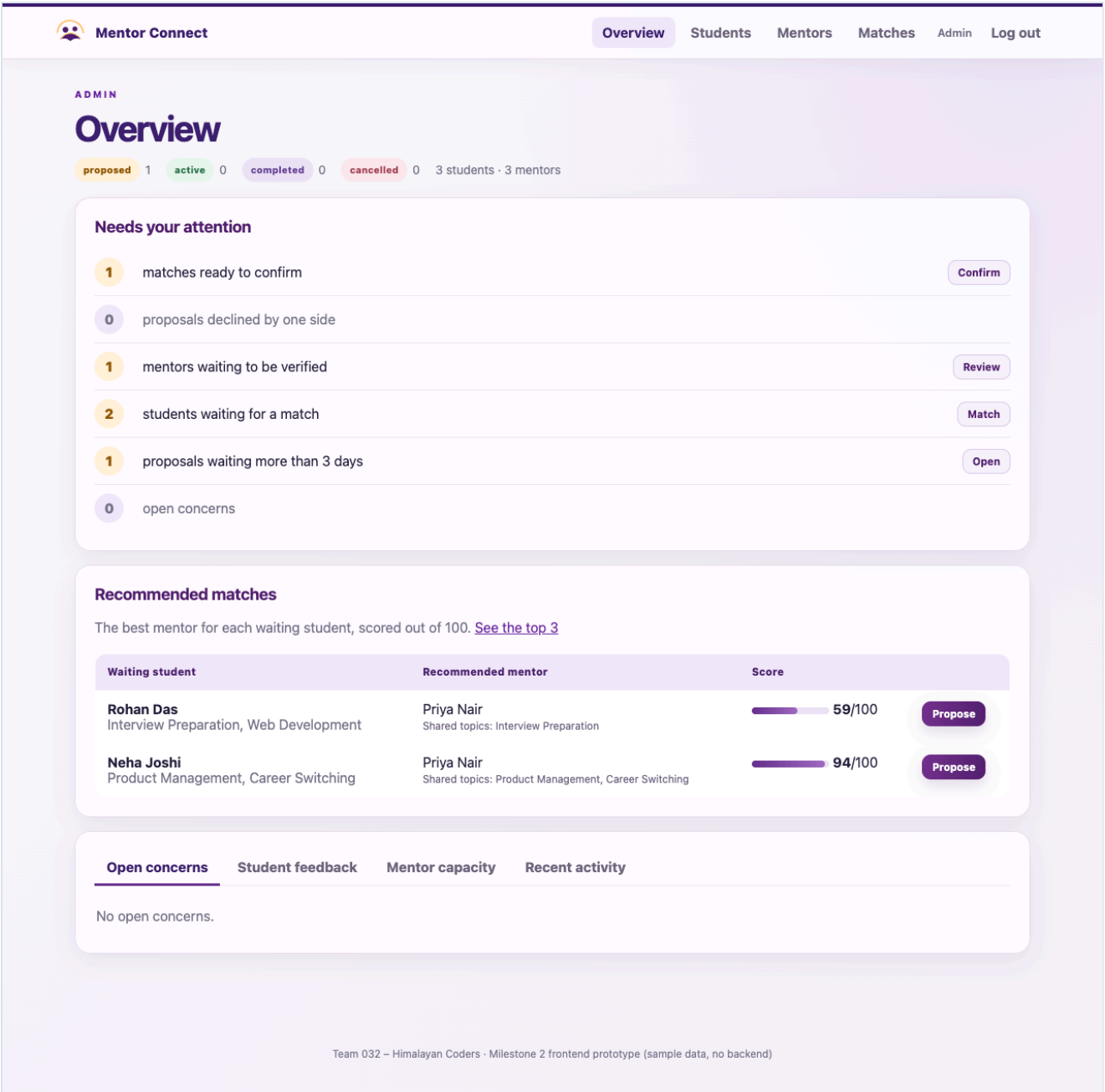


Figure 9. Admin overview: what needs attention, the recommended matches, and tabs for concerns, feedback, capacity and activity.

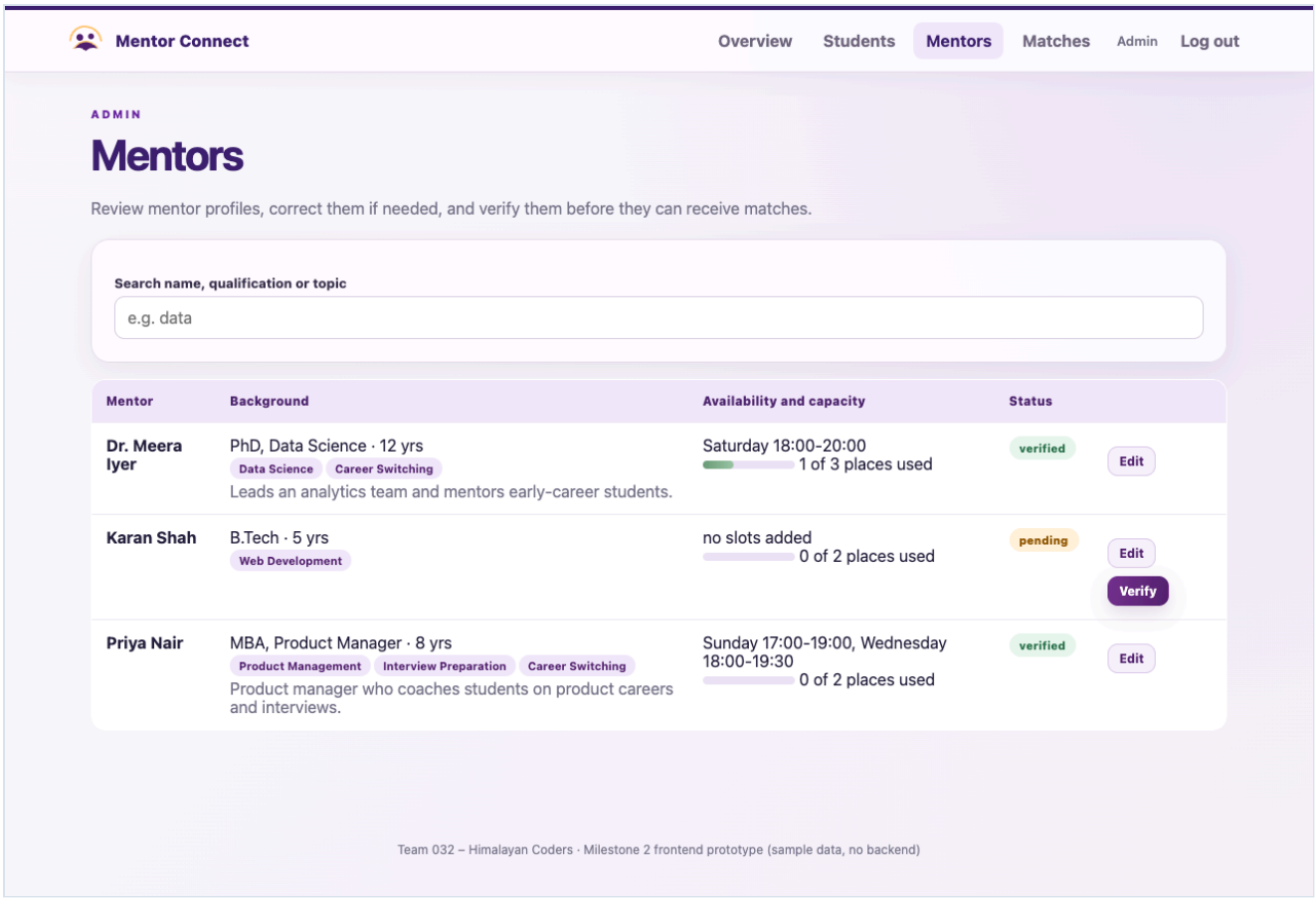


Figure 10. Mentor review with topics, availability and capacity.

Mentor Connect | Overview | Students | Mentors | Matches | Admin | Log out

ADMIN

Matches

Recommended matches

Each waiting student is scored against every verified mentor with free places, out of 100: topics 70 and mentor experience 30. Mentors who are full are not shown. A score under 40 is a weak fit. The platform only suggests; you decide.

Waiting student	Recommended mentor	Score	Why	
Rohan Das Interview Preparation, Web Development	Priya Nair Product Management, Interview Preparation, Career Switching	**59**/100	Shared topics: Interview Preparation 8 years of experience 2 of 2 places free	Propose
	Dr. Meera Iyer Data Science, Career Switching	**30**/100 weak fit	No shared topics 12 years of experience 2 of 3 places free	Propose
Neha Joshi Product Management, Career Switching	Priya Nair Product Management, Interview Preparation, Career Switching	**94**/100	Shared topics: Product Management, Career Switching 8 years of experience 2 of 2 places free	Propose
	Dr. Meera Iyer Data Science, Career Switching	**65**/100	Shared topics: Career Switching 12 years of experience 2 of 3 places free	Propose

Propose a match yourself

Both the mentor and the student are notified. You can confirm the match only after both have accepted, or you can cancel it.

Student without a match

Rohan Das

Verified mentor

Dr. Meera Iyer (1 of 3 places used)

Student needs

Goals
Prepare for software interviews

Topics wanted
Interview Preparation, Web Development

Skills
HTML, JavaScript

Format and duration
Messaging · 1 month

Availability
Sunday afternoons

Mentor offer

Qualifications
PhD, Data Science · 12 yrs

Topics covered
Data Science, Career Switching

Bio
Leads an analytics team and mentors early-career students.

Availability
Saturday 18:00-20:00

Capacity
1 of 3 places used

Match score: **30**/100 topics 0/70 · experience 30/30
No shared topics · 12 years of experience · 2 of 3 places free
Weak fit. You can still propose it.

Notes (the basis for this match)

Why this pairing?

Propose match

All matches

Student	Mentor	Status	Score	Mentor response	Student response	Proposed	Action
Asha Verma	Dr. Meera Iyer	proposed 1 update(s)	**100**/100	pending	pending	29 Oct 2026	Confirm match Cancel

Figure 11. Matches page with the top three mentors for each waiting student, the score of a chosen pair, and the list of matches.

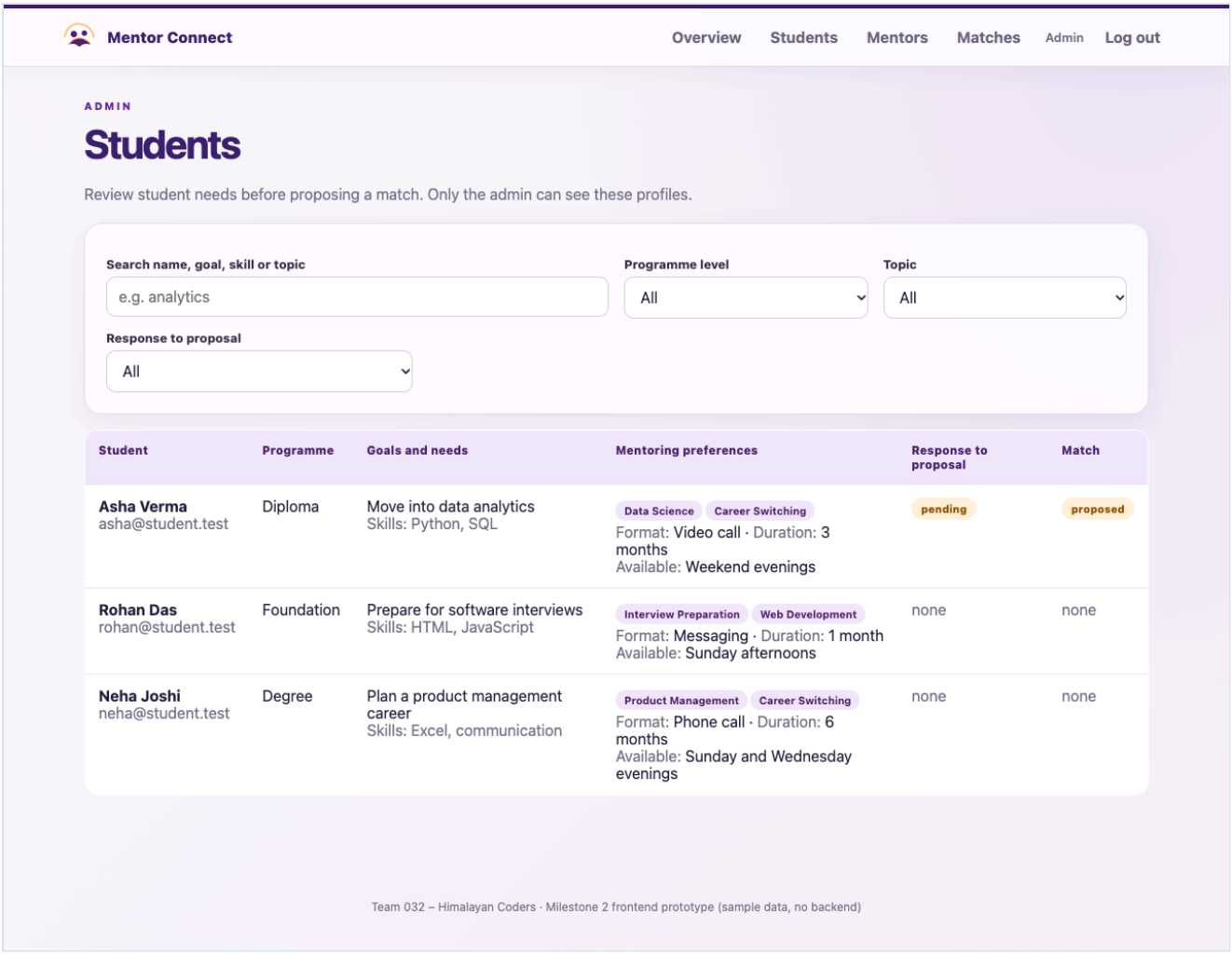


Figure 12. Student profiles with search and filters.

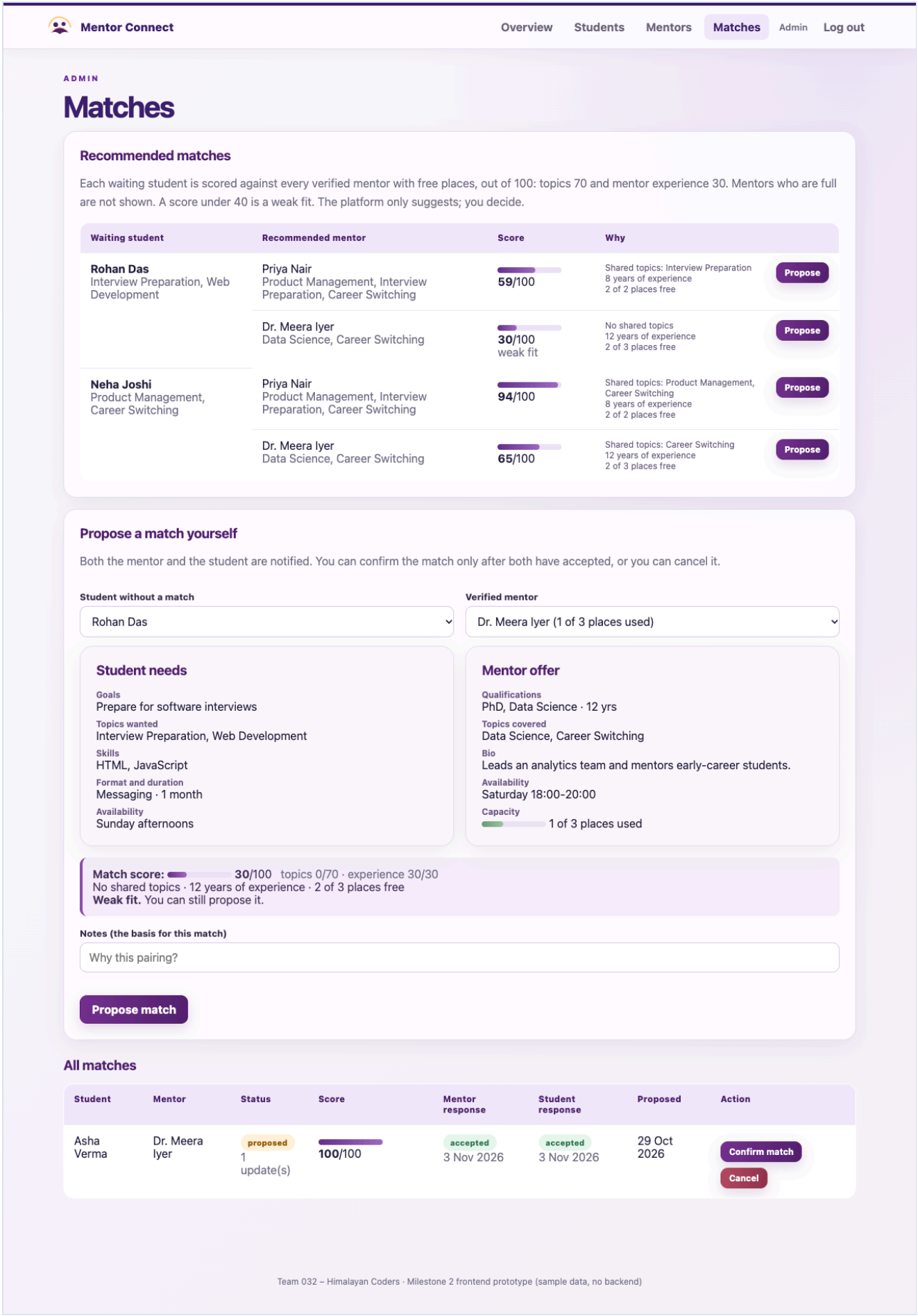


Figure 13. After both sides accept, the Confirm match button is enabled. Before that it is disabled.

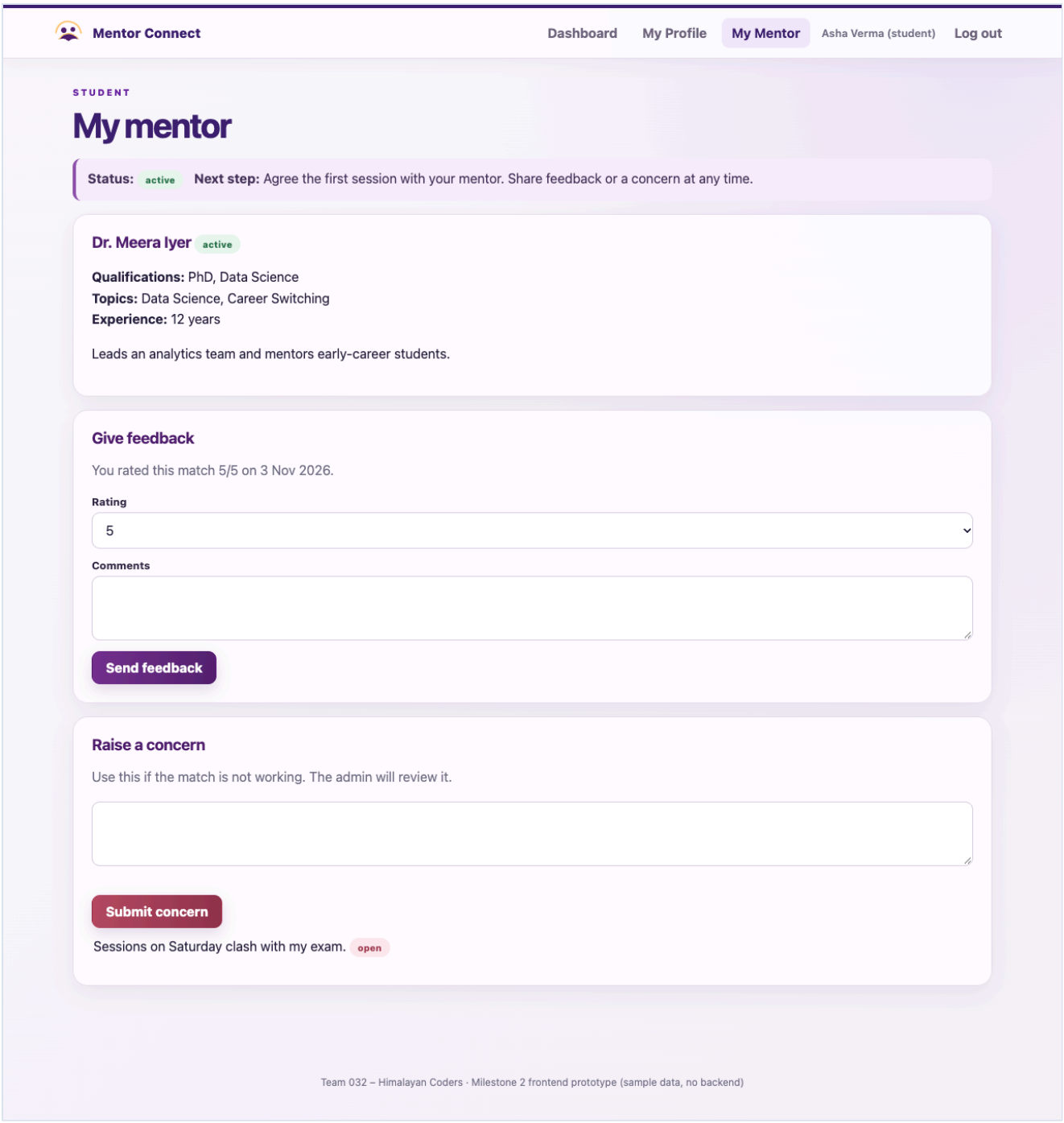


Figure 14. Student's mentor page after the admin confirms: status, next step, mentor details, feedback and concern forms.

7. Frontend Code and README

The frontend ZIP has the Vue project: the pages (`src/views`), shared components, the data layer (`src/services`), the styles, unit tests and a README. To run it you need Node.js 18 or newer. Unzip, open the `frontend` folder, then run `npm install` and `npm run dev`, and open the address shown. The password for the demo accounts (student, mentor and admin) is `demo123`. The accounts are listed on the login page. `npm run build` makes the files that a host such as Vercel serves, and `npm test` runs the unit tests.

Folder	What is in it
views	One file for each page, grouped as auth, student, mentor and admin.
components	Small shared parts: header, status badges, score and capacity bars, topic picker, recommendation table, comparison cards.
services	The data layer: login, matching and scores, match flow, profiles, feedback, notifications. Replaced by API calls in Sprint 1.
router, layouts	Page addresses with the role guard, and the two page layouts.
config, data	Topics, statuses and score weights in one place, and the sample data.

8. Open Questions and Risks

- What should happen after a mentor or a student declines? In the prototype the admin decides, cancels the match and can propose a new one.
- How are notifications sent, and is email needed?
- How long is data kept, and who may see feedback?
- Should the system set a maximum number of students per mentor, and who keeps the list of topics?
- Does the client agree with the score weights (topics 70, experience 30)?